Project Planning and Management

MediCare - Project Proposal, Plan, Roles, Risk, and KPIs

DEPI Full Stack .NET Track | Submission deadline: 16 October 2026

1. Project Proposal

Project overview

MediCare is a web-based clinic management and appointment system that connects patients with doctors and helps clinics coordinate schedules and authorized medical information. The application follows a 3-layer N-Tier Architecture (MediCare.Web, MediCare.Services, MediCare.Data) and applies the Repository Pattern, Unit of Work Pattern, Factory Pattern, and Dependency Injection. It utilizes ASP.NET Core MVC, Entity Framework Core, ASP.NET Identity, SignalR (real-time notifications), Twilio (SMS), SendGrid (Email), FullCalendar.js, Chart.js, Bootstrap 5, and Microsoft Azure.

Objectives

- Help patients find doctors by specialization, location, and availability.
- Reduce appointment conflicts through availability checks and server-side validation.
- Enable doctors to manage schedules and review relevant patient information.
- Provide patients with appointment history and access to authorized records and prescriptions.
- Support appointment confirmations, notifications, and reminders where configured.
- Give administrators account, doctor, appointment, and activity-management tools.
- Improve clinic coordination and provide a basis for measuring service activity.

Project scope

In scope	Out of scope for this graduation project
• Multi-role authentication (Patient, Doctor, Admin via ASP.NET Identity) • N-Tier Architecture with Repository / Unit of Work / Factory patterns • FullCalendar.js booking with conflict detection • Doctor working hours & request approval • Electronic Medical Records with X-ray/test result uploads • Printable digital prescriptions • SignalR real-time alerts • Twilio SMS & SendGrid Email 24h reminders • Admin Chart.js analytics with PDF/Excel export • Bootstrap 5 Dark/Light mode • Microsoft Azure deployment	Diagnosis or clinical decision support; emergency care; payments and insurance claims; live hospital-system integration; production interoperability; real patient data in development or demonstrations.

Project plan

Suggested eight-week schedule. Replace relative weeks with actual project start dates. The final milestone is aligned to the supplied submission deadline.

Workstream	W1	W2	W3	W4	W5	W6	W7	W8
Requirements and stakeholder review	●	●						
UI, database and architecture design		●	●					
Authentication and role dashboards			●	●				
Doctor search and profiles				●				
Availability and appointments				●	●			
Records, notifications and admin					●	●		
Integration and defect fixing						●	●	
Testing, docs and presentation							●	●

Milestones and deliverables

Milestone	Target	Evidence
Milestone 1 (Planning, Literature Review & Requirements)	16 Oct 2026	Approved proposal, plan, literature review, stakeholders, stories, use cases, and FR/NFRs
Milestone 2 (System Analysis & Design)	06 Nov 2026	Architecture, ERD, schema, UML diagrams (Use Case, DFD, Sequence, Activity, State, Class, Deployment), and wireframes
Milestone 3 (Implementation - Source Code & Execution)	30 Nov 2026	Complete N-Tier source code, booking, dashboards, records, notifications, admin analytics, and Azure deployment
Milestone 4 (Final Presentation, Testing & Reports)	04 Dec 2026	Test plan/reports, user manual, project infographic, presentation, and final defense

Task assignment and roles

Proposed allocation based on the team's instructions: Ali and Sara handle frontend; the other three members handle backend. Adjust ownership to match skills and availability.

Team member	Area	Assigned Modules & Responsibilities
Ali Mohamed Mohamed Saleh	Frontend	Patient Portal UI, Doctor Search/Filter Views, FullCalendar.js Integration, Printable Prescription UI & Use Cases Docs.
Sara Elsaieed Zidan Mahmoud	Frontend	Doctor & Admin Dashboards UI, Chart.js Analytics Views, Bootstrap 5 Dark/Light Mode, Evaluation & Docs Review.
Basma Mohamed Abdelmageed Mohamed	Backend	Doctor & Appointment Repositories, Schedule/Working Hours Logic, Conflict Detection, SignalR Real-Time Alerts & Project Planning Docs.
Youssef Mohamed Saeed Alazraq	Backend	N-Tier Architecture Setup, Generic Repository & Unit of Work, ASP.NET Identity Multi-Role Auth, Azure CI/CD & Literature Review Docs.
Amr Nasser Khalifa Makhlouf	Backend	Medical Records & Document Uploads, Digital Prescriptions, Factory Pattern with Twilio SMS & SendGrid Email, PDF/Excel Export & FR/NFR Docs.

Resource allocation

Resource	Purpose
Five team members and supervisor	Requirements decisions, implementation, reviews, and acceptance feedback.
ASP.NET Core MVC and .NET SDK	Application framework and server-side implementation.
SQL Server & Entity Framework Core (with Repository and Unit of Work patterns)	Persistence for users, doctors, schedules, appointments and records.
Bootstrap 5, JavaScript, FullCalendar.js, Chart.js	Responsive views, calendar interactions and dashboard visualizations.
SignalR, Twilio, and SendGrid	Real-time in-app notifications and automated 24-hour SMS/Email appointment reminders.
GitHub & Microsoft Azure	Version control, CI/CD pipelines, and cloud deployment.
Git and task board	Version control, task ownership, code review and defect tracking.
Test environment	Local/demo build with synthetic data; no real patient data.

Risk assessment and mitigation

Risk	Likelihood / impact	Mitigation
Race condition permits double booking.	Medium / High	Recheck availability on submission and use a transaction or database constraint; test simultaneous requests.
Unauthorized medical-data access.	Medium / High	Use role and ownership checks, least privilege, secure session practices, and access-control tests.
Scope exceeds available time.	High / Medium	Protect an MVP; defer optional uploads, advanced analytics, and integrations until booking works.
Cancellation or record-access policy is unclear.	Medium / High	Record as an open decision and get team/supervisor agreement before implementation.
Notification provider unavailable.	Medium / Medium	Keep delivery replaceable; use in-app or mock notifications for the demo and disclose limitations.
Integration conflict or schedule slippage.	Medium / Medium	Integrate weekly, use small reviewed changes, document interfaces, and assign backup owners.
Unsafe or insufficient test data.	Low / High	Use synthetic data and keep identifiable health information out of repos, screenshots, and demos.

Key Performance Indicators

Targets are proposed acceptance goals, not measured results. Record the test environment, sample size, date, and calculation for each KPI.

KPI	Measurement	Proposed target
Search response time	Median response for doctor-search requests under agreed load.	≤ 2 seconds
Booking response time	Submission-to-confirmation/conflict response.	≤ 2 seconds
Conflict prevention	Conflicting bookings accepted ÷ attempted conflicting bookings.	0 accepted conflicts in tests
Demo availability	Available monitored time during scheduled demo/test window.	≥ 99% in that window
Core workflow tests	Passed core tests ÷ executed core tests.	≥ 95%; no open critical defects
Usability completion	Representative users completing core tasks unaided.	≥ 80% in walkthrough
Notification success	Accepted messages ÷ attempted test messages.	≥ 95% in test environment
Adoption	Active users completing a core workflow during a pilot.	Set after pilot; report funnel

Measurement note

A coursework prototype has no production uptime or adoption history. Report actual demo observations separately from proposed targets.